**Fundamentos de Programación Web**

**Tarea 1:**

Cuestionario

**Autor:**

Armando Arias Alvarado

**Profesor:**

Francisco Jiménez Bonilla

Febrero, 2026

1. **Escriba la historia de Java Script.**

JavaScript fue creado en Netscape Communications por Brendan Eich en 1995. Netscape y Eich lo diseñaron como lenguaje de scripting para su uso con el navegador web estrella de la compañía, Netscape Navigator. Inicialmente conocido como LiveScript, Netscape cambió el nombre a JavaScript para posicionarlo como complemento del lenguaje Java, un producto de su socio, Sun Microsystems. Sin embargo, salvo algunas similitudes sintácticas superficiales, JavaScript no guarda ninguna relación con el lenguaje de programación Java.

NETSCAPE®

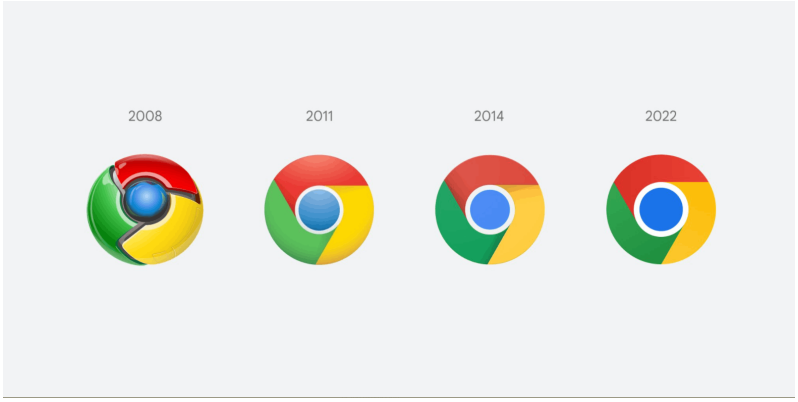
*Logo de Netscape Navigator.*

*Logo de Java.*

Tras su lanzamiento, cada vez más navegadores empezaron a añadir compatibilidad con JavaScript. Aun así, durante gran parte de su historia, JavaScript no se consideró un lenguaje de programación serio. Sus primeras versiones presentaban importantes problemas de rendimiento y seguridad, pero los desarrolladores no tenían alternativas. Si querían ejecutar programas en el navegador, tenían que usar JavaScript.

*Logo de JavaScript.*

En 2008, la creación de Chrome V8 de código abierto de Google, un motor JavaScript de alto rendimiento, marcó un punto de inflexión crucial para JavaScript. La posterior proliferación de motores JavaScript rápidos permitió a los desarrolladores crear sofisticadas aplicaciones basadas en navegador con un rendimiento que competía con las aplicaciones de escritorio y móviles.



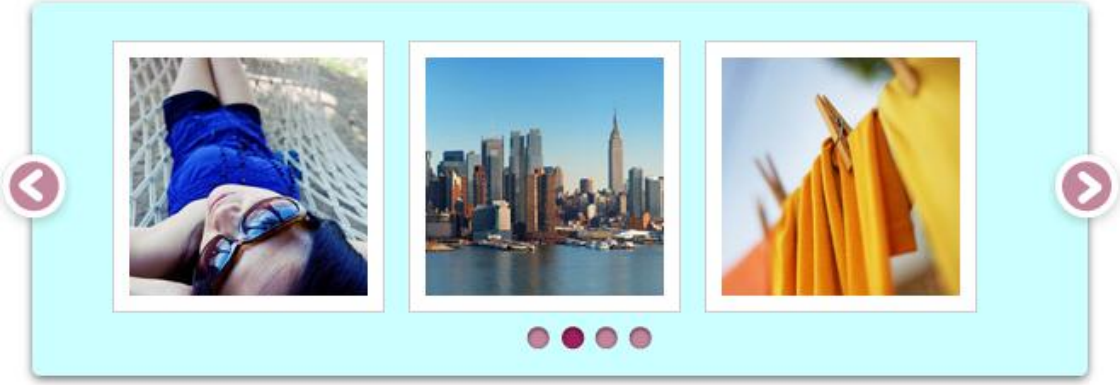

*Logo de Chrome a través de los años.*

Poco después, Ryan Dahl lanzó un entorno multiplataforma de código abierto llamado Node.js. Este entorno permitía ejecutar código JavaScript desde fuera del navegador. Liberó a JavaScript de las limitaciones del navegador y contribuyó directamente a su popularidad actual. Hoy en día, se puede usar JavaScript para crear todo tipo de aplicaciones, incluyendo aplicaciones de navegador, servidor, móviles y de escritorio.

*Logo de Node.js.*

## 2. ¿Qué es Java Script?

JavaScript es un lenguaje de programación que los desarrolladores utilizan para hacer páginas web interactivas. Desde actualizar fuentes de redes sociales a mostrar animaciones y mapas interactivos, las funciones de JavaScript pueden mejorar la experiencia del usuario de un sitio web. Como lenguaje de scripting del lado del servidor, se trata de una de las principales tecnologías de la World Wide Web. Por ejemplo, al navegar por Internet, en cualquier momento en el que vea un carrusel de imágenes, un menú desplegable “click-to-show” (clic para mostrar), o cambien de manera dinámica los elementos de color en una página web, estará viendo los efectos de JavaScript.



*Ejemplo de un carrusel de imágenes.*

3. **¿Cuál es la relación entre HTML y Java Script?**

HTML, CSS y JavaScript son parte integral de la creación de una página web funcional y visualmente atractiva. HTML proporciona el contenido y la estructura, CSS añade el diseño y el estilo, y JavaScript aporta elementos interactivos. Trabajan juntos para dar vida a un sitio web.

Sin embargo, es importante tener en cuenta que mientras el HTML y el CSS son necesarios para que un sitio web funcione, el JavaScript no siempre es necesario. Hay ciertos elementos que se pueden conseguir solo con HTML y CSS, como los efectos hover o el diseño responsivo. Todo depende de las necesidades y objetivos específicos del sitio web.

*Logo de HTML.*

HTML, CSS y Javascript son lenguajes de programación que solos funcionan bien, pero juntos pueden darle a un proyecto otro nivel. Algunos ejemplos de usos de los tres lenguajes juntos:

- Una imagen en una página web se define y se le da su tamaño con HTML. Se utiliza CSS para añadir un borde y cambiar la posición de la imagen en la página. A continuación, se puede utilizar JavaScript para crear un efecto de animación cuando el usuario pasa por encima de la imagen.

- Un formulario de contacto en una página web se crea con HTML, se estiliza con CSS, y se hace funcional con JavaScript para enviar la información introducida a la dirección de correo electrónico apropiada.

- Un menú de navegación en una página web se crea con HTML y se estiliza con CSS. A continuación, se puede utilizar JavaScript para añadir un efecto desplegable cuando el usuario pasa por encima de un elemento del menú.

- Un mapa interactivo en una página web se crea con HTML, se estiliza con CSS, y se hace funcional con JavaScript para mostrar información diferente o resaltarla cuando el usuario clics en un área específica.

- Las páginas de productos de una tienda online utilizan HTML para mostrar los artículos, CSS para el estilo y el diseño, y JavaScript para añadir elementos interactivos como un carrito de la compra dinámico o un carrusel de imágenes de productos.

*Logo de CSS.*

## 4. ¿En qué beneficia usar Bootstrap para sitios y aplicaciones web en JS?

- Fácil de usar: cualquiera con conocimientos básicos de HTML y CSS puede comenzar a usar Bootstrap.
- Funciones de respuesta: el CSS responsivo de Bootstrap se ajusta a teléfonos, tabletas y computadoras de escritorio.
- Enfoque móvil primero: en Bootstrap, los estilos móviles primero son parte del marco central.
- Compatibilidad con navegadores: Bootstrap 5 es compatible con todos los navegadores modernos (Chrome, Firefox, Edge, Safari y Opera).

*Logo de Bootstrap.*

5. **¿Qué semejanza y diferencia tienen los lenguajes web PHP y Java Script?**

| PHP | JavaScript |
|---|---|
| Scripting del lado del servidor | Scripting del lado del cliente |
| Usado en backend | Usado en el frontend (ahora con Node.js) |
| Solo se combina con HTML | Se combina con múltiples lenguajes |
| Parcialmente sensible a las mayúsculas y minúsculas | Completamente sensible a las mayúsculas y minúsculas |
| Diferencias de sintaxis, por ejemplo, el # está permitido para los comentarios | Diferencias de sintaxis, por ejemplo, el # no está permitido para los comentarios |
| Las variables declaradas con el prefijo $ | Las variables declaradas con var o let las palabras clave |
| Tiene conjuntos asociativos | No tiene conjuntos asociativos |
| Se integra con muchas bases de datos | Apoyo deficiente o inexistente de la base de datos |
| Multi-hilo | Un solo hilo |
| Rápido con PHP 7.0 o superior | Normalmente es más rápido que el PHP |
| Utiliza los gestores de paquetes PEAR y Composer | Utiliza npm, hilo y gestores de paquetes de Bower |

| | |
|---|---|
| Rápido de ejecutar si la versión de PHP es > 7.x | Generalmente más rápido que PHP |
| Utilizado en alrededor del 80% de los sitios web | Utilizado en casi todos los sitios web |

6. **¿Cite 3 formas en que se puede agregar código JS en una página web?**

1. Añadir Javascript en las etiquetas <head>

El código JS puede ser insertado dentro de la sección <head> del documento HTML. Esto se hace para mantener la función contenida y alejada del contenido principal del documento HTML.

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>

<h2>Demo JavaScript in Head</h2>

<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>

</body>
</html>
```

2. Añadir Javascript en las etiquetas <Body>

Si el script debe ejecutarse en un punto específico dentro del diseño de la página web, deberá ser insertado en el punto donde debe llamar a la función. La colocación suele ser dentro de la sección <body>.

```html
<!DOCTYPE html>
<html>
<body>

<h2>Demo JavaScript in Body</h2>

<p id="demo">A Paragraph</p>

<button type="button" onclick="myFunction()">Try it</button>

<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>

</body>
</html>
```

3. Crear un archivo JavaScript externo

Si el script es grande, o necesita repartirse en varias páginas web, lo normal es añadir el código JS a uno o varios archivos JavaScript a los que se hace referencia dentro de los documentos HTML.

El uso de archivos JS separados permite a los desarrolladores desagregar el marcado de HTML del código JS, evitando cualquier confusión. También facilita el mantenimiento continuo y permite que las páginas web se carguen más rápidamente al almacenar en caché los archivos JS.

External file: myScript.js

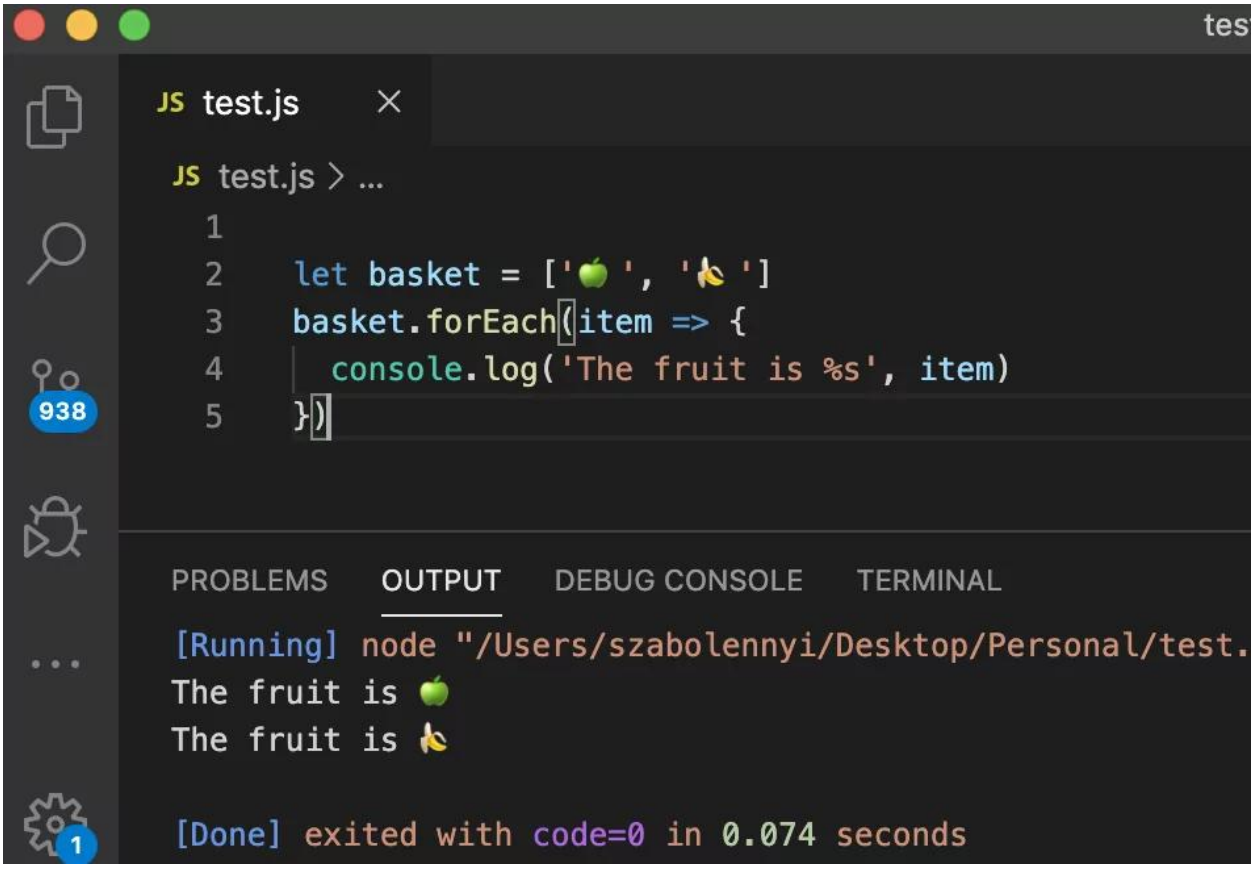
```javascript
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
```

Example

```html
<script src="myScript.js"></script>
```

7. **¿Cuál es la función principal de la consola en JS?**

La función principal de la consola en JavaScript es ser una herramienta interactiva para depurar código, ejecutar fragmentos de JavaScript, y mostrar mensajes, errores y advertencias durante el desarrollo. Funciona como un bucle REPL (Read-Eval-Print Loop), permitiendo a los desarrolladores escribir, evaluar e imprimir resultados de código JavaScript en un entorno interactivo.




*Uso de la consola en JavaScript.*

8. **¿Cuál es la diferencia que existe en las declaraciones var, let y const en JS? ¿Brinde un ejemplo de cada una utilizando una tabla?**

Al declarar variables en JavaScript, hay tres opciones: var, let y const. Aunque las tres palabras clave permiten declarar variables, tienen diferentes alcances y comportamientos.

**Var**

Var es la palabra clave original utilizada para declarar variables en JavaScript, y ha existido desde que se introdujo el lenguaje por primera vez. También es la más flexible de las tres palabras clave, ya que permite declarar la misma variable varias veces y reasignarle valores.

Una de las desventajas de var es que tiene ámbito de función. Esto significa que una variable declarada con var está disponible en toda la función en la que se declara. Si declaras una variable dentro de un bloque o bucle, sigue estando disponible fuera de ese bloque o bucle, lo que puede dar lugar a resultados inesperados al intentar reutilizar nombres de variables o al trabajar con funciones anidadas.

**Let**

Let es una palabra clave relativamente nueva introducida en ES6. Se introdujo para resolver algunos de los problemas asociados con var. La principal diferencia entre let y var es que let tiene ámbito de bloque. Esto significa que una variable declarada con let sólo está disponible dentro del bloque en el que se declara. Si se declara una variable dentro de un bloque o bucle, no estará disponible fuera de ese bloque o bucle.

Otra diferencia entre let y var es que no puedes declarar la misma variable varias veces utilizando let. Esto puede ayudar a prevenir conflictos de nombres y hacer tu código más legible.

**Const**

Const es otra palabra clave introducida en ES6, y se utiliza para declarar constantes. Una constante es un valor que no puede ser reasignado una vez que ha sido declarado. Esto puede ser útil cuando se trabaja con valores que nunca deben cambiar, como las constantes matemáticas o las opciones de configuración.

Una cosa importante para tener en cuenta sobre const es que no hace que el valor de la variable sea inmutable. Si declaras un objeto o un array usando const, todavía puedes modificar las propiedades de ese objeto o array. Sin embargo, no puedes reasignar la propia variable.

| Característica | var | let | const |
|---|---|---|---|
| ¿Dónde funciona? | En toda la función | Solo dentro de las llaves {} | Solo dentro de las llaves {} |
| ¿Puedo declararla otra vez? | Sí | No | No |
| ¿Puedo cambiarle el valor? | Sí | Sí | No |
| ¿Funciona antes de declararla? | Sí (pero sale undefined) | No (da error) | No (da error) |
| ¿Necesita valor inicial? | No | No | Sí |

## 9. ¿Cuál es la función de minificar archivos Java Script?

La minificación también se conoce como minimización. La minificación de código significa optimizar el código para ahorrar espacio, reducir los tiempos de carga de la página y disminuir el uso del ancho de banda del sitio web. Sin embargo, la mayor preocupación es minimizar el código sin alterar la funcionalidad.

La minificación del código es posible en todas las tecnologías de programación principales, incluyendo HTML, CSS y JavaScript. Sin embargo, el proceso no es instantáneo. Hay que trabajar un poco para hacer el código más compacto sin que deje de ser funcional.

Para minificar el código JavaScript, debes analizarlo, comprimirlo y obtener el resultado. Una vez minificado, debería ser casi ilegible a simple vista. Has eliminado todos los espacios en blanco innecesarios, los comentarios, los caracteres de nueva línea y todo lo que inicialmente hacía legible el código.

Es posible que tengas que hacer algunos cambios más en el código, por ejemplo, alinear funciones, eliminar delimitadores de bloque, utilizar condicionales implícitos o reescribir variables locales.

```
// program to compare two arrays

function compareArrays(arr1, arr2) {

    // compare arrays
    const result = JSON.stringify(arr1) == JSON.stringify(arr2)

    // if result is true
    if(result) {
        console.log('The arrays have the same elements.');
    }
    else {
        console.log('The arrays have different elements.');
    }

}

const array1 = [1, 3, 5, 8];
const array2 = [1, 3, 5, 8];

compareArrays(array1, array2);
```

Select Language: JS CSS

Minify

```
function compareArrays(arr1,arr2){const result=JSON.stringify(arr1)==JSON.stringify(arr2)
if(result){console.log('The arrays have the same elements.')}else{console.log('The arrays have different elements.')}}
const array1=[1,3,5,8];const array2=[1,3,5,8];compareArrays(array1,array2)
```

*Ejemplo minificación de código JS.*

10. **¿Qué es ECMAScript6? Explique claramente.**

ES6 o ECMAScript 6 es un estándar de lenguaje de scripting desarrollado de acuerdo con las indicaciones contenidas en el documento ECMA-262 proporcionado por ECMA International (European Computer Manufacturers Association). ECMA International es una asociación fundada en 1961 que se ocupa de la estandarización en el sector de las tecnologías de la información y los sistemas de comunicación.

ECMAScript es, por lo tanto, un estándar creado con el objetivo de permitir la interoperabilidad de diferentes páginas web entre distintos navegadores. El estándar de lenguaje especifica las características que debe tener un lenguaje de scripting y cómo deben implementarse.

**ECMAScript 6**

ES y Javascript a lo largo de los años han lanzado sucesivas versiones de su producto, en concreto al analizar ES6 vemos como la sexta versión se ha marcado los siguientes objetivos:

- Desarrollo de aplicaciones complejas
- Creación y gestión de bibliotecas mejoradas
- Mejor generación de código

ES6, también conocido como ECMAScript 2015 o simplemente "Harmony" es uno de los estándares más importantes. Dada su importancia, el browser support creció rápidamente desde Edge y Safari hasta Chrome, Firefox y Opera y hoy es compatible con todos los navegadores.

**Novedades sobre ECMAScript 6**

Las novedades de esta nueva versión son numerosas e importantes como se mencionó anteriormente. Nos encontramos con la implementación de nuevas palabras claves, funciones y propiedades que realmente han revolucionado el lenguaje.

- Keyword “let”, permite modificar el valor de una variable al interno de un espacio delimitado para luego dar nuevamente la variable al valor precedente.
- Keyword "const", muy similar a la keyword "let" pero el valor asignado a la variable no puede cambiar dentro de los paréntesis en la que son declarados. Externamente recupera el valor inicial.
- Las nuevas arrow functions que permiten la declaración de funciones en modo más eficaz ahorrando las líneas del código.
- Introducción del ciclo for/of para ser utilizado con Array, String etc.
- Keyword “class” para la creación de clases y constructos.
- Nuevos objetos como los “promise” que se conectan a los “producing code” que pueden requerir un poco de tiempo a los “consuming code” que deben esperar el resultado.
- El Symbol Type, un Symbol en Javascript es un tipo de dato primitivo y representa un identificador escondido al que ningún otro código puede acceder, a menudo viene utilizado por los ID.
- ES6 permite a las funciones de tener los parámetros con valores de default.
- La función Rest Parameter que trata un indefinido número de argumentos como un array.
- El Array.find. La función find() torna el valor del primer elemento del array que corresponde a los criterios indicados.
- Agrega nuevos métodos a los objetos matemáticos:

Math.trunc() devuelve la parte entera del número.

Math.sign() devuelve si el número es positivo, negativo o neutro.

Math.cbrt() devuelve la raíz cúbica del número.

Math.log2() devuelve el logaritmo en base dos del algoritmo.

Math.log10() devuelve el algoritmo en base diez del argumento.

- Adición de nuevas constantes matemáticas: EPSILON, MIN_SAFE_INTEGER e MAX_SAFE_INTEGER.
- Number.isInteger() devuelve True si el argumento es un número interno.
- isNaN(), devuelve true si el argumento no es un número.
- Moduli: Soporte de lenguaje de módulo para crear y administrar componentes: para crear un módulo solo se necesita escribir algo de código JavaScript en un file para exportar una o más de las funciones implementadas. La funcionalidad y los otros elementos exportados de un módulo JavaScript pueden ser importante de un otro módulo trámite la palabra clave import.
- Notamos una mejor gestión de la parte matemática y una renovada gestión de arrays y strings, así como la introducción de keyword orientadas a simplificar el trabajo del programador y reducir las líneas de código.

## Conclusión

La realización de este cuestionario permitió consolidar el conocimiento sobre JavaScript, desde sus orígenes hasta sus aplicaciones modernas. Se comprendió que JavaScript, lejos de ser solo un lenguaje para añadir interactividad básica en el navegador, es una tecnología robusta y versátil que impulsa el desarrollo tanto en el front-end como en el back-end, gracias a entornos como Node.js.

Quedaron claras las diferencias esenciales entre los tipos de declaración de variables (var, let, const), un concepto importante para escribir código predecible y libre de errores. Además, se pudo apreciar la sinergia entre HTML, CSS y JavaScript como los tres pilares del desarrollo web front-end, donde cada uno cumple una función específica pero complementaria.

La investigación sobre herramientas y conceptos avanzados como Bootstrap, la minificación de código y el estándar ECMAScript 6 (ES6) mostró la evolución constante y la importancia de seguir mejores prácticas para crear aplicaciones eficientes, mantenibles y con buen desempeño.

En resumen, este trabajo sirvió para sentar una base sólida y comprender que JavaScript es un lenguaje en constante evolución, cuyo dominio es indispensable para cualquier desarrollador web moderno. El aprendizaje obtenido proporciona el marco de referencia necesario para continuar explorando frameworks, librerías y conceptos más avanzados con confianza.

**Referencias Web**


Amazon Web Services, Inc. (s.f.). *¿Qué es JavaScript? - Una introducción al JavaScript*. AWS. Recuperado el 3 de febrero de 2026 de https://aws.amazon.com/es/what-is/javascript/

Dongee. (s.f.). *¿Cómo se relaciona HTML, CSS y JavaScript?*. Recuperado el 3 de febrero de 2026 de https://www.dongee.com/tutoriales/como-se-relaciona-html-css-y-javascript/

Kinsta. (2023, 28 noviembre). *PHP vs JavaScript: ¿Cuál elegir en 2024?*. Recuperado el 3 de febrero de 2026 de https://kinsta.com/es/blog/php-vs-javascript/

Launch School. (s.f.). *Introducción a JavaScript*. Recuperado el 3 de febrero de 2026 de https://launchschool.com/books/javascript/read/introduction

Skillnest. (s.f.). *¿Cómo añadir JavaScript a HTML en 2 sencillos pasos?*. Recuperado el 3 de febrero de 2026 de https://www.skillnest.com/blog/como-anadir-javacript-a-html-en-2-sencillos-pasos/

W3Schools. (s.f.). *Bootstrap 5 Get Started*. Recuperado el 3 de febrero de 2026 de https://www.w3schools.com/bootstrap5/bootstrap_get_started.php

AulaB. (2023, 25 octubre). *¿Qué es ECMAScript 6?*. Recuperado el 4 de febrero de 2026 de https://aulab.es/noticia/17/que-es-ecmascript-6

Chrome Developers. (s.f.). *Ejecutar JavaScript en la consola*. Recuperado el 4 de febrero de 2026 de https://developer.chrome.com/docs/devtools/console/javascript?hl=es-419

Kinsta. (2024, 8 febrero). *Cómo minificar CSS, HTML y JavaScript*. Recuperado el 4 de febrero de 2026 de https://kinsta.com/es/blog/minificar-javascript/

Medium. (2023, 18 julio). *JavaScript: ¿Cuáles son las diferencias entre var, let y const?*. Wink Hosting Latam. Recuperado el 4 de febrero de 2026 de https://medium.com/winkhostinglatam/javascript-cu%C3%A1les-son-las-diferencias-entre-var-let-y-const-711c23408be7